

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №4

з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Оцінка складності алгоритму імітації

1. Текст завдання:

1. Розробити модель масового обслуговування, яка складається з N послідовно з'єднаних систем масового обслуговування. Число N є параметром моделі. Кількість подій в моделі оцінюється числом N + 1. 20 балів.
2. Виконати експериментальну оцінку складності алгоритму імітації мережі масового обслуговування. Для цього виконайте серію експериментів, в якій спостерігається збільшення часу обчислення алгоритму імітації при збільшенні кількості подій в моделі. 20 балів.
3. Виконати теоретичну оцінку складності побудованого алгоритму імітації. 20 балів.
4. Повторіть експериментальну та теоретичну оцінку складності алгоритму імітації при зміні структури мережі масового обслуговування: замість послідовно з'єднаних систем масового обслуговування розгляньте розгалужені маршрути слідування між системами масового обслуговування. Зробіть висновок про вплив розгалужень маршруту на складність алгоритму. 40 балів.

2. Код програми:

Програму написано мовою Python 3.12. Основою є універсальний алгоритм імітації з практикуму № 3 (файли `fun_rand.py`, `item.py`, `element.py`, `elements.py`, `model.py`); у нього додано лічильник подій і повідомлення про зміну моменту наступної події елемента, потрібне для алгоритму з календарем подій. Вихідний код і результати розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab4`). Нові файли:

- `networks.py` – мережі з N систем масового обслуговування: послідовний ланцюг, бінарне дерево, паралельні гілки, ланцюг з поверненням;
- `fast_model.py` – алгоритм імітації з календарем подій у вигляді двійкової купи;
- `complexity.py` – експерименти, оцінка моделей вартості та графіки.

Файл fun_rand.py

```
import math
import random
```

```
_rng = random.Random()


def seed(value):
    _rng.seed(value)


def exp(time_mean):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return -time_mean * math.log(a)


def unif(time_min, time_max):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return time_min + a * (time_max - time_min)


def norm(time_mean, time_deviation):
    return _rng.gauss(time_mean, time_deviation)


def erlang(time_mean, k):
    product = 1.0
    for _ in range(k):
        product *= 1.0 - _rng.random()
    return -time_mean / k * math.log(product)


def uniform01():
    return _rng.random()
```

Файл item.py

```
class Item:
    """Вимога, що рухається моделлю: зберігає тип, момент створення та довільні атрибути."""

    _next_id = 0

    def __init__(self, t_create, kind=1, **attrs):
        self.id = Item._next_id
        Item._next_id += 1
        self.t_create = t_create
        self.kind = kind
        self.t_queue = t_create
        self.attrs = attrs
```

Файл element.py

```
import math

import fun_rand


class Route:
    def __init__(self, target, probability=1.0, priority=0, condition=None):
        self.target = target
        self.probability = probability
        self.priority = priority
        self.condition = condition
        self.count = 0


class Element:
    """Базовий елемент моделі.

    Затримка задається числом (середнє значення закону distribution) або функцією item -> затримка.
    Маршрут виходу обирається за ймовірністю ("probability"), за пріоритетом ("priority": перший за
    пріоритетом блок, що може прийняти вимогу) або функцією routing(item, routes) -> route.
    Умова маршруту condition(item) дозволяє розводити вимоги за типом.
    """

    _next_id = 0
```

```
def __init__(self, name=None, delay=1.0, delay_dev=0.0, distribution="exp", routing="probability"):
    self.id = Element._next_id
    Element._next_id += 1
    self.name = name or f"element{self.id}"
    self.delay = delay
    self.delay_dev = delay_dev
    self.distribution = distribution
    self.routing = routing
    self.routes = []
    self.on_enter = None
    self.listener = None
    self.tnext = math.inf
    self.tcurr = 0.0
    self.state = 0
    self.quantity = 0

def add_route(self, target, probability=1.0, priority=0, condition=None):
    self.routes.append(Route(target, probability, priority, condition))
    return self

def can_accept(self):
    return True

def _before_change(self):
    if self.listener:
        self.listener.before(self)

def _after_change(self):
    if self.listener:
        self.listener.after(self)

def choose_route(self, item):
    candidates = [r for r in self.routes if r.condition is None or r.condition(item)]
    if not candidates:
        return None
    if callable(self.routing):
        route = self.routing(item, candidates)
    elif self.routing == "priority":
        ordered = sorted(candidates, key=lambda r: -r.priority)
        route = next((r for r in ordered if r.target.can_accept()), ordered[0])
    else:
        z = fun_rand.uniform01() * sum(r.probability for r in candidates)
        route = candidates[-1]
        cumulative = 0.0
        for r in candidates:
            cumulative += r.probability
            if z < cumulative:
                route = r
                break
    route.count += 1
    return route.target

def send(self, item):
    target = self.choose_route(item)
    if target is not None:
        target.in_act(item)

def get_delay(self, item=None):
    if callable(self.delay):
        return self.delay(item)
    kind = self.distribution.lower()
    if kind == "exp":
        return fun_rand.exp(self.delay)
    if kind == "norm":
        return max(0.0, fun_rand.norm(self.delay, self.delay_dev))
    if kind == "unif":
        return fun_rand.unif(self.delay - self.delay_dev, self.delay + self.delay_dev)
    if kind == "erlang":
        return fun_rand.erlang(self.delay, int(self.delay_dev))
    return self.delay

def in_act(self, item):
    pass

def out_act(self):
    self.quantity += 1
```

```
    def do_statistics(self, delta):
        pass

    def reset_statistics(self):
        self.quantity = 0

    def print_info(self):
        print(f"{self.name} state = {self.state} quantity = {self.quantity} tnext = {self.tnext:.4f}")
```

Файл elements.py

```
import math
from collections import defaultdict

from element import Element
from item import Item


class Create(Element):
    def __init__(self, delay, first_time=0.0, item_factory=None, **kwargs):
        super().__init__(delay=delay, **kwargs)
        self.tnext = first_time
        self.item_factory = item_factory or (lambda t: Item(t))

    def out_act(self):
        super().out_act()
        item = self.item_factory(self.tcurr)
        self.tnext = self.tcurr + self.get_delay(item)
        self._after_change()
        self.send(item)


class Process(Element):
    """Багатоканальний пристрій з чергою обмеженої довжини.

    channels може бути math.inf - тоді елемент моделює затримку без обмеження кількості вимог
    (наприклад, переміщення). queue_key(item) задає пріоритет вибору з черги: першою обслуговується
    вимога з найменшим ключем, за рівних ключів - та, що прийшла раніше.
    """

    def __init__(self, delay, channels=1, maxqueue=math.inf, queue_key=None, record_arrivals=False, **kwargs):
        super().__init__(delay=delay, **kwargs)
        self.channels = channels
        self.maxqueue = maxqueue
        self.queue_key = queue_key
        self.record_arrivals = record_arrivals
        self.in_service = []
        self.queue = []
        self.reset_statistics()

    def reset_statistics(self):
        super().reset_statistics()
        self.failure = 0
        self.arrivals = 0
        self.queue_area = 0.0
        self.busy_area = 0.0
        self.wait = defaultdict(lambda: [0.0, 0])
        self.arrival_times = []

    @property
    def busy(self):
        return len(self.in_service)

    def can_accept(self):
        return self.busy < self.channels or len(self.queue) < self.maxqueue

    def _refresh(self):
        self.tnext = min((t for t, _ in self.in_service), default=math.inf)
        self.state = self.busy
        self._after_change()

    def _start(self, item):
        record = self.wait[item.kind]
        record[0] += self.tcurr - item.t_queue
        record[1] += 1
        self.in_service.append((self.tcurr + self.get_delay(item), item))
```

```
    def accept(self, item):
        """Ставить вимогу на обслуговування або в чергу без перевірки обмеження черги (переведення вимог)."""
        item.t_queue = self.tcurr
        if self.busy < self.channels:
            self._start(item)
        else:
            self.queue.append(item)
        self._refresh()

    def in_act(self, item):
        self._before_change()
        if self.on_enter:
            self.on_enter(item)
        self.arrivals += 1
        if self.record_arrivals:
            self.arrival_times.append(self.tcurr)
        if self.can_accept():
            self.accept(item)
        else:
            self.failure += 1

    def _take_from_queue(self):
        if self.queue_key is None:
            return self.queue.pop(0)
        index = min(range(len(self.queue)), key=lambda i: (self.queue_key(self.queue[i]), i))
        return self.queue.pop(index)

    def out_act(self):
        self._before_change()
        finished = [item for t, item in self.in_service if t == self.tcurr]
        self.in_service = [(t, item) for t, item in self.in_service if t != self.tcurr]
        for _ in finished:
            self.quantity += 1
            if self.queue and self.busy < self.channels:
                self._start(self._take_from_queue())
        self._refresh()
        for item in finished:
            self.send(item)

    def do_statistics(self, delta):
        self.queue_area += len(self.queue) * delta
        self.busy_area += self.busy * delta

    def mean_queue(self, total_time):
        return self.queue_area / total_time

    def mean_load(self, total_time):
        """Частка часу зайнятості каналів; для необмеженої кількості каналів – середня кількість зайнятих."""
        channels = 1 if self.channels == math.inf else self.channels
        return self.busy_area / (total_time * channels)

    def mean_wait(self, kind=None):
        records = [self.wait[kind]] if kind is not None else list(self.wait.values())
        total, count = sum(r[0] for r in records), sum(r[1] for r in records)
        return total / count if count else 0.0

    def print_info(self):
        print(f"{self.name} state = {self.state} queue = {len(self.queue)} quantity = {self.quantity} "
              f"failure = {self.failure} tnext = {self.tnext:.4f}")


class Dispose(Element):
    """Вихід з моделі: фіксує моменти виходу й час перебування вимог за групами group(item)."""

    def __init__(self, name="DISPOSE", group=lambda item: item.kind):
        super().__init__(name=name)
        self.group = group
        self.stats_start = 0.0
        self.reset_statistics()

    def reset_statistics(self):
        super().reset_statistics()
        self.exit_times = []
        self.times = defaultdict(list)

    def in_act(self, item):
```

```
        self.quantity += 1
        self.exit_times.append(self.tcurr)
        if item.t_create >= self.stats_start:
            self.times[self.group(item)].append(self.tcurr - item.t_create)

    def all_times(self):
        return [t for values in self.times.values() for t in values]
```

Файл model.py

```
import math
import statistics

from elements import Create, Dispose, Process


class Model:
    """Імітація за принципом найближчої події.

    hooks – функції hook(model), які викликаються після кожної події (правила, що змінюють стан кількох
    елементів одночасно). warmup – тривалість розгону: накопичена до цього моменту статистика скидається.
    """

    def __init__(self, elements, hooks=()):
        self.elements = elements
        self.hooks = list(hooks)
        self.tcurr = 0.0
        self.stats_start = 0.0
        self.in_system_area = 0.0
        self.events = 0

    def processes(self):
        return [e for e in self.elements if isinstance(e, Process)]

    def in_system(self):
        return sum(len(p.queue) + p.busy for p in self.processes())

    def _accumulate(self, delta):
        for e in self.elements:
            e.do_statistics(delta)
        self.in_system_area += self.in_system() * delta

    def _reset(self, moment):
        self.stats_start = moment
        self.in_system_area = 0.0
        for e in self.elements:
            e.reset_statistics()
            if isinstance(e, Dispose):
                e.stats_start = moment

    def simulate(self, time, warmup=0.0, trace_events=0):
        while self.tcurr < time:
            tnext = min(e.tnext for e in self.elements)
            t_stop = min(tnext, time)
            if self.tcurr < warmup <= t_stop:
                self._accumulate(warmup - self.tcurr)
                self._reset(warmup)
                self._accumulate(t_stop - warmup)
            else:
                self._accumulate(t_stop - self.tcurr)
            if tnext > time:
                self.tcurr = time
                break

            self.tcurr = tnext
            for e in self.elements:
                e.tcurr = self.tcurr
            active = [e for e in self.elements if e.tnext == self.tcurr]
            for e in active:
                e.out_act()
            for hook in self.hooks:
                hook(self)

            self.events += 1
            if self.events <= trace_events:
                names = ", ".join(e.name for e in active)
                print(f"\nIt's time for event in {names}, time = {self.tcurr:.4f}")
```

```
                for e in self.elements:
                    e.print_info()
        for e in self.elements:
            e.tcurr = self.tcurr

    def results(self):
        total = self.tcurr - self.stats_start
        disposes = [e for e in self.elements if isinstance(e, Dispose)]
        times = [t for d in disposes for t in d.all_times()]
        exits = sorted(t for d in disposes for t in d.exit_times)
        out = {
            "time": total,
            "created": sum(e.quantity for e in self.elements if isinstance(e, Create)),
            "disposed": sum(d.quantity for d in disposes),
            "failures": sum(p.failure for p in self.processes()),
            "mean_in_system": self.in_system_area / total,
            "mean_time_in_system": statistics.mean(times) if times else math.nan,
            "mean_exit_interval": (exits[-1] - exits[0]) / (len(exits) - 1) if len(exits) > 1 else math.nan,
            "processes": {},
            "groups": {},
        }
        for p in self.processes():
            out["processes"][p.name] = {
                "arrivals": p.arrivals, "served": p.quantity, "failure": p.failure,
                "mean_queue": p.mean_queue(total), "mean_load": p.mean_load(total),
                "mean_wait": p.mean_wait(),
                "wait_by_kind": {k: v[0] / v[1] for k, v in p.wait.items() if v[1]},
            }
        for d in disposes:
            for group, values in d.times.items():
                out["groups"][f"{d.name}: {group}"] = statistics.mean(values)
        return out
```

Файл fast_model.py

```
import heapq
import itertools
import math

from elements import Process
from model import Model


class FastModel(Model):
    """Алгоритм імітації з календарем подій у вигляді двійкової купи.

    Момент найближчої події береться з вершини купи за O(1), а не пошуком серед усіх елементів;
    елемент, у якого змінився момент наступної події, додається в купу за O(log N). Статистику
    накопичує лише елемент, стан якого змінюється, – за час від попередньої зміни його стану.
    """

    def __init__(self, elements):
        super().__init__(elements)
        self.calendar = []
        self.planned = {}
        self.last_change = {}
        self.order = itertools.count()
        for e in elements:
            e.listener = self
            self.last_change[e] = 0.0
            self.after(e)

    def before(self, element):
        element.tcurr = self.tcurr
        element.do_statistics(self.tcurr - self.last_change[element])
        self.last_change[element] = self.tcurr

    def after(self, element):
        if element.tnext != math.inf and self.planned.get(element) != element.tnext:
            self.planned[element] = element.tnext
            heapq.heappush(self.calendar, (element.tnext, next(self.order), element))

    def simulate(self, time, warmup=0.0, trace_events=0):
        while self.calendar:
            t, _, element = self.calendar[0]
            if self.planned.get(element) != t:
                heapq.heappop(self.calendar)  # застарілий запис: момент події елемента вже змінився
```

```
                continue
            if t > time:
                break
            heapq.heappop(self.calendar)
            del self.planned[element]
            self.tcurr = t
            element.tcurr = t
            element.out_act()
            self.after(element)
            self.events += 1
        self.tcurr = time
        for e in self.elements:
            self.before(e)

    def results(self):
        out = super().results()
        processes = [e for e in self.elements if isinstance(e, Process)]
        out["mean_in_system"] = sum(p.queue_area + p.busy_area for p in processes) / self.tcurr
        return out
```

Файл networks.py

```
"""Мережі масового обслуговування з N системами різної структури."""
import math

from elements import Create, Dispose, Process
from model import Model

ARRIVAL_MEAN, SERVICE_MEAN = 1.0, 0.3
FEEDBACK = 0.5


def _assemble(nodes, create, dispose, engine):
    return engine([create, *nodes, dispose])


def chain(n, engine=Model):
    """N послідовно з'єднаних систем: кожна вимога проходить усі N систем."""
    create, dispose = Create(ARRIVAL_MEAN, name="CREATE"), Dispose()
    nodes = [Process(SERVICE_MEAN, name=f"P{i + 1}") for i in range(n)]
    create.add_route(nodes[0])
    for current, following in zip(nodes, nodes[1:] + [dispose]):
        current.add_route(following)
    return _assemble(nodes, create, dispose, engine)


def tree(n, engine=Model):
    """Бінарне дерево з N = 2^d - 1 систем: після кожної системи вимога з ймовірністю 0,5 йде в одну
    з двох наступних, після листа - виходить з моделі."""
    create, dispose = Create(ARRIVAL_MEAN, name="CREATE"), Dispose()
    nodes = [Process(SERVICE_MEAN, name=f"P{i + 1}") for i in range(n)]
    create.add_route(nodes[0])
    for i, node in enumerate(nodes):
        children = [j for j in (2 * i + 1, 2 * i + 2) if j < n]
        if children:
            for j in children:
                node.add_route(nodes[j], probability=1 / len(children))
        else:
            node.add_route(dispose)
    return _assemble(nodes, create, dispose, engine)


def parallel(n, engine=Model):
    """N паралельних систем: вимога з однаковою ймовірністю потрапляє в одну з них і виходить."""
    create, dispose = Create(ARRIVAL_MEAN, name="CREATE"), Dispose()
    nodes = [Process(SERVICE_MEAN, name=f"P{i + 1}") for i in range(n)]
    for node in nodes:
        create.add_route(node, probability=1 / n)
        node.add_route(dispose)
    return _assemble(nodes, create, dispose, engine)


def feedback(n, engine=Model):
    """Ланцюг з N систем з поверненням: після останньої системи вимога з ймовірністю 0,5
    повертається в першу."""
    create, dispose = Create(ARRIVAL_MEAN, name="CREATE"), Dispose()
    nodes = [Process(SERVICE_MEAN, name=f"P{i + 1}") for i in range(n)]
```

```
    create.add_route(nodes[0])
    for current, following in zip(nodes, nodes[1:]):
        current.add_route(following)
    nodes[-1].add_route(dispose, probability=1 - FEEDBACK)
    nodes[-1].add_route(nodes[0], probability=FEEDBACK)
    return _assemble(nodes, create, dispose, engine)


def path_length(structure, n):
    """Середня кількість систем, які проходить одна вимога."""
    return {
        "chain": n,
        "tree": math.log2(n + 1),
        "parallel": 1,
        "feedback": n / (1 - FEEDBACK),
    }[structure]


def sojourn_time(structure, n):
    """Середній час проходження вимоги через мережу в сталому режимі (кожна система – M/M/1)."""
    rho = SERVICE_MEAN / ARRIVAL_MEAN
    if structure == "feedback":
        rho /= 1 - FEEDBACK
    return path_length(structure, n) * SERVICE_MEAN / (1 - rho)


STRUCTURES = {"chain": chain, "tree": tree, "parallel": parallel, "feedback": feedback}
TITLES = {"chain": "послідовний ланцюг", "tree": "бінарне дерево", "parallel": "паралельні гілки",
          "feedback": "ланцюг з поверненням"}
```

Файл complexity.py

```
"""Завдання 1-4: модель з N послідовних СМО, експериментальна і теоретична оцінка складності
алгоритму імітації для послідовної та розгалужених структур мережі."""
import json
import math
import platform
import sys
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

import fun_rand
from fast_model import FastModel
from model import Model
from networks import ARRIVAL_MEAN, STRUCTURES, TITLES, chain, path_length, sojourn_time

RESULTS = Path(__file__).resolve().parent / "results"
TIME = 500.0
REPEATS = 3
SIZES = {
    Model: {"chain": [5, 10, 20, 50, 100, 150, 200], "feedback": [5, 10, 20, 50, 100, 150, 200],
            "parallel": [5, 10, 20, 50, 100, 150, 200], "tree": [3, 7, 15, 31, 63, 127, 255]},
    FastModel: {"chain": [5, 10, 20, 50, 100, 200, 500, 1000],
                "feedback": [5, 10, 20, 50, 100, 200, 500, 1000],
                "parallel": [5, 10, 20, 50, 100, 200, 500, 1000],
                "tree": [3, 7, 15, 31, 63, 127, 255, 511, 1023]},
}
ENGINE_NAMES = {Model: "базовий алгоритм", FastModel: "календар подій"}
COLORS = {"chain": "#2c5d8f", "feedback": "#c0392b", "tree": "#27ae60", "parallel": "#8e44ad"}


def demo():
    """Модель з N = 5 послідовних СМО: трасування перших подій і результати."""
    fun_rand.seed(1)
    n = 5
    model = chain(n)
    print(f"Модель з N = {n} послідовних СМО: надходження exp({ARRIVAL_MEAN}), обслуговування exp(0.3), "
          f"час моделювання 1000")
    model.simulate(1000.0, trace_events=3)
    r = model.results()
    print("\n-------------RESULTS-------------")
    for name, p in r["processes"].items():
```

```
        print(f"{name}: обслуговано {p['served']}, завантаження {p['mean_load']:.4f}, "
              f"черга {p['mean_queue']:.4f}")
    print(f"створено {r['created']}, вийшло {r['disposed']}, "
          f"середній час перебування {r['mean_time_in_system']:.4f}")
    print(f"кількість подій {model.events}; оцінка λ·T·(N + 1) = {1000 / ARRIVAL_MEAN * (n + 1):.0f}")
    return {"n": n, "events": model.events, "created": r["created"], "disposed": r["disposed"],
            "time_in_system": r["mean_time_in_system"],
            "loads": [p["mean_load"] for p in r["processes"].values()]}


def measure(structure, n, engine):
    """Час обчислення відрізка модельного часу TIME у сталому режимі: спочатку без заміру часу
    виконується розгін тривалістю у два середні часи проходження мережі."""
    warmup = 2 * sojourn_time(structure, n) + 10
    best = math.inf
    for repeat in range(REPEATS):
        fun_rand.seed(100 + repeat)
        model = STRUCTURES[structure](n, engine)
        model.simulate(warmup)
        events_before = model.events
        start = time.perf_counter()
        model.simulate(warmup + TIME)
        best = min(best, time.perf_counter() - start)
    return {"structure": structure, "engine": ENGINE_NAMES[engine], "n": n, "elements": n + 2,
            "events": model.events - events_before, "seconds": best, "path": path_length(structure, n),
            "warmup": warmup, "expected_events": TIME / ARRIVAL_MEAN * (1 + path_length(structure, n))}


def fit_cost(rows, feature):
    """МНК для моделі t = E · (c0 + c1 · feature(N)): вартість однієї події лінійна за feature."""
    e = np.array([r["events"] for r in rows], dtype=float)
    f = np.array([feature(r["n"]) for r in rows], dtype=float)
    t = np.array([r["seconds"] for r in rows])
    # відносна похибка важливіша за абсолютну, тому рівняння діляться на t
    a = np.column_stack([e, e * f]) / t[:, None]
    (c0, c1), *_ = np.linalg.lstsq(a, np.ones_like(t), rcond=None)
    predicted = e * (c0 + c1 * f)
    for r, p in zip(rows, predicted):
        r["predicted"] = float(p)
    return float(c0), float(c1), float(np.max(np.abs(predicted - t) / t))


def slope(rows):
    """Показник степеня b у залежності t ~ N^b за трьома найбільшими N."""
    tail = sorted(rows, key=lambda r: r["n"])[-3:]
    x = np.log([r["n"] for r in tail])
    y = np.log([r["seconds"] for r in tail])
    return float(np.polyfit(x, y, 1)[0])


def plot(rows, engine_name, filename, title):
    fig, axes = plt.subplots(1, 3, figsize=(14, 4.3))
    for structure in STRUCTURES:
        data = sorted((r for r in rows if r["structure"] == structure and r["engine"] == engine_name),
                      key=lambda r: r["n"])
        n = [r["n"] for r in data]
        color = COLORS[structure]
        axes[0].plot(n, [r["seconds"] for r in data], "o-", color=color, label=TITLES[structure])
        axes[0].plot(n, [r["predicted"] for r in data], "--", color=color, linewidth=1)
        axes[1].plot(n, [r["events"] for r in data], "o-", color=color, label=TITLES[structure])
        axes[2].plot(n, [r["seconds"] / r["events"] * 1e6 for r in data], "o-", color=color,
                     label=TITLES[structure])
    for ax in axes[:2]:
        ax.set_xscale("log")
        ax.set_yscale("log")
    axes[2].set_xlim(left=0)
    axes[0].set_ylabel("час обчислення, с")
    axes[1].set_ylabel(f"кількість подій за T = {TIME:.0f}")
    axes[2].set_ylabel("час на одну подію, мкс")
    for ax in axes:
        ax.set_xlabel("кількість СМО N")
        ax.grid(alpha=0.3, which="both")
        ax.legend(fontsize=8)
    fig.suptitle(title)
    fig.tight_layout()
    fig.savefig(RESULTS / filename, dpi=200)
    plt.close(fig)
```

```
def plot_engines(rows):
    fig, ax = plt.subplots(figsize=(8, 4.5))
    for engine_name, style in (("базовий алгоритм", "o-"), ("календар подій", "s--")):
        data = sorted((r for r in rows if r["structure"] == "chain" and r["engine"] == engine_name),
                      key=lambda r: r["n"])
        ax.plot([r["n"] for r in data], [r["seconds"] for r in data], style, label=engine_name)
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("кількість СМО N")
    ax.set_ylabel("час обчислення, с")
    ax.set_title("Послідовний ланцюг: базовий алгоритм і календар подій")
    ax.grid(alpha=0.3, which="both")
    ax.legend()
    fig.tight_layout()
    fig.savefig(RESULTS / "engines.png", dpi=200)
    plt.close(fig)


def main():
    RESULTS.mkdir(exist_ok=True)
    summary = {"demo": demo(), "time": TIME, "repeats": REPEATS,
               "python": sys.version.split()[0], "platform": platform.platform()}
    rows = []
    for engine, sizes in SIZES.items():
        for structure, values in sizes.items():
            for n in values:
                row = measure(structure, n, engine)
                rows.append(row)
                print(f"{row['engine']:17} {structure:9} N = {n:5}: подій {row['events']:8} "
                      f"(оцінка {row['expected_events']:9.0f}), час {row['seconds']:8.4f} с, "
                      f"{row['seconds'] / row['events'] * 1e6:6.2f} мкс/подію", flush=True)

    base = [r for r in rows if r["engine"] == "базовий алгоритм"]
    fast = [r for r in rows if r["engine"] == "календар подій"]
    summary["base_fit"] = dict(zip(("c0", "c1", "max_error"), fit_cost(base, lambda n: n + 2)))
    # у паралельній структурі на кожне надходження додається вибір одного з N маршрутів, тому її
    # вартість оцінюється окремо: t = E · c0 + λT · c1 · N
    summary["fast_fit"] = dict(zip(("c0", "c1", "max_error"),
                                   fit_cost([r for r in fast if r["structure"] != "parallel"],
                                            lambda n: math.log2(n + 2))))
    parallel = [r for r in fast if r["structure"] == "parallel"]
    summary["fast_parallel_fit"] = dict(zip(("c0", "c1", "max_error"),
                                            fit_cost(parallel, lambda n: n / 2)))
    summary["slopes"] = {engine: {s: slope([r for r in rows if r["engine"] == engine and r["structure"] == s])
                                  for s in STRUCTURES} for engine in ENGINE_NAMES.values()}
    summary["rows"] = rows

    print("\nМодель вартості базового алгоритму: t = E · (c0 + c1 · (N + 2)), "
          f"c0 = {summary['base_fit']['c0'] * 1e6:.3f} мкс, c1 = {summary['base_fit']['c1'] * 1e9:.2f} нс, "
          f"макс. відносна похибка {summary['base_fit']['max_error'] * 100:.1f} %")
    print("Модель вартості з календарем подій (ланцюг, дерево, повернення): t = E · (c0 + c1 · log2(N + 2)), "
          f"c0 = {summary['fast_fit']['c0'] * 1e6:.3f} мкс, c1 = {summary['fast_fit']['c1'] * 1e9:.2f} нс, "
          f"макс. відносна похибка {summary['fast_fit']['max_error'] * 100:.1f} %")
    pf = summary["fast_parallel_fit"]
    print("Модель вартості з календарем подій (паралельні гілки): t = E · (c0 + c1 · N / 2), "
          f"c0 = {pf['c0'] * 1e6:.3f} мкс, c1 = {pf['c1'] * 1e9:.2f} нс, "
          f"макс. відносна похибка {pf['max_error'] * 100:.1f} %")
    for engine, values in summary["slopes"].items():
        print(f"Показник степеня t ~ N^b ({engine}): " +
              ", ".join(f"{TITLES[s]} {b:.2f}" for s, b in values.items()))

    plot(rows, "базовий алгоритм", "complexity_base.png", "Базовий алгоритм імітації (пошук найближчої події "
         "перебором)")
    plot(rows, "календар подій", "complexity_fast.png", "Алгоритм з календарем подій (двійкова купа)")
    plot_engines(rows)
    with open(RESULTS / "complexity.json", "w", encoding="utf-8") as f:
        json.dump(summary, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

3. Результати роботи програми:

Завдання 1:

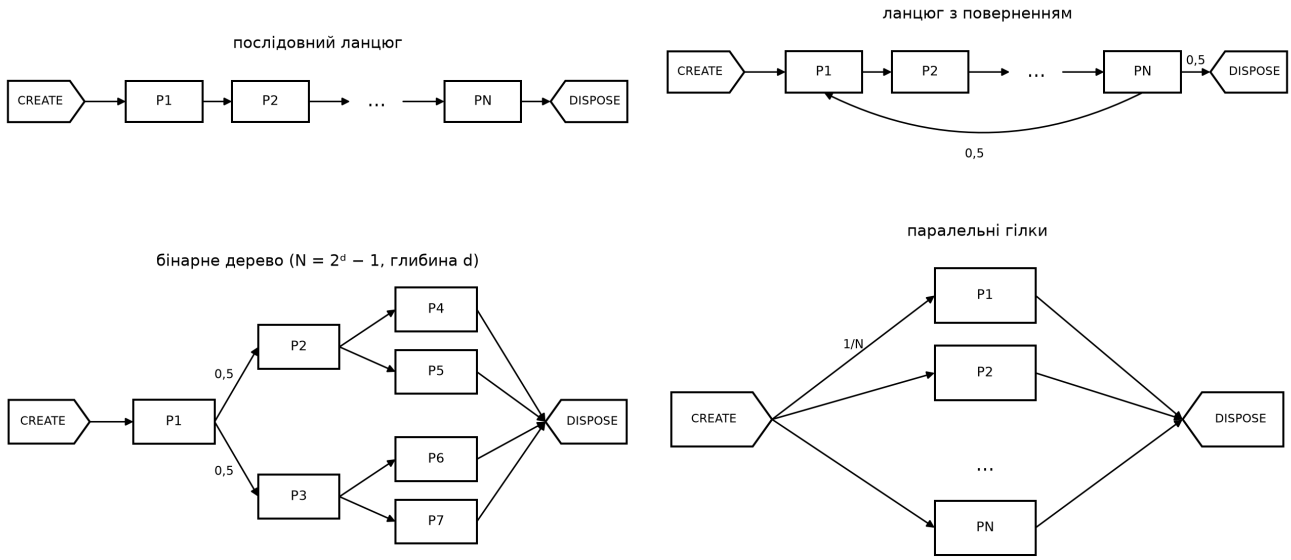


Рисунок 1 – Структури мереж масового обслуговування з N систем

Модель (функція `chain` у файлі `networks.py`, рис. 1, ліворуч угорі) складається з елемента CREATE, N послідовно з'єднаних одноканальних систем P1…PN з необмеженою чергою та елемента DISPOSE. Число N передається параметром, і модель будується циклом. Інтервал між надходженнями exp(1,0), час обслуговування в кожній системі exp(0,3), тому завантаження кожної системи ρ = 0,3 і мережа стаціонарна за будь-якого N. Подій у моделі N + 1: надходження вимоги та закінчення обслуговування в кожній з N систем. Фрагмент виводу для N = 5:

```
Модель з N = 5 послідовних СМО: надходження exp(1.0), обслуговування exp(0.3), час моделювання
1000

It's time for event in CREATE, time = 0.0000
CREATE state = 0 quantity = 1 tnext = 2.0072
P1 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.0808
P2 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
P3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
P4 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
P5 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in P1, time = 0.0808
CREATE state = 0 quantity = 1 tnext = 2.0072
P1 state = 0 queue = 0 quantity = 1 failure = 0 tnext = inf
P2 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.2915
P3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
P4 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
P5 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

...

-------------RESULTS-------------
P1: обслуговано 989, завантаження 0.3024, черга 0.1425
P2: обслуговано 988, завантаження 0.2897, черга 0.1153
P3: обслуговано 986, завантаження 0.2913, черга 0.1145
P4: обслуговано 985, завантаження 0.3096, черга 0.1145
P5: обслуговано 984, завантаження 0.2896, черга 0.1009
створено 990, вийшло 984, середній час перебування 2.0958
кількість подій 5922; оцінка λ·T·(N + 1) = 6000
```

За 1000 одиниць часу відбулося 5922 подій. Кожна вимога породжує одну подію надходження та N подій закінчення обслуговування, тому очікувана кількість подій $\lambda \cdot T \cdot (N + 1) = 1000 \cdot 6 = 6000$; завантаження всіх систем близьке до 0,3.

Завдання 2:

Для кожного N модель моделювалася протягом T = 500 одиниць модельного часу і вимірювався процесорний час виконання методу `simulate` (функція `time.perf_counter`). Щоб замір відповідав сталому режиму, перед ним без заміру часу виконувався розгін тривалістю у два середні часи проходження вимоги через мережу: інакше за великих N вимоги не встигають пройти весь ланцюг і подій стає менше, ніж у сталому режимі. Кожен замір повторено 3 рази, у таблиці наведено найменший час (найменше спотворений фоновими процесами). Обчислення виконано на Apple M1, Python 3.12.

Таблиця 1 – Час обчислення моделі з N послідовних СМО

N	Подій E	Оцінка λT(1 + L)	Час, с	Час на подію, мкс	Модель вартості, с
5	2927	3000	0,0145	4,96	0,0140
10	5120	5500	0,0322	6,29	0,0313
20	10396	10500	0,0926	8,91	0,0916
50	25501	25500	0,4143	16,25	0,4306
100	51216	50500	1,4362	28,04	1,5538
150	77172	75500	3,4017	44,08	3,3795
200	100649	100500	5,4823	54,47	5,7618

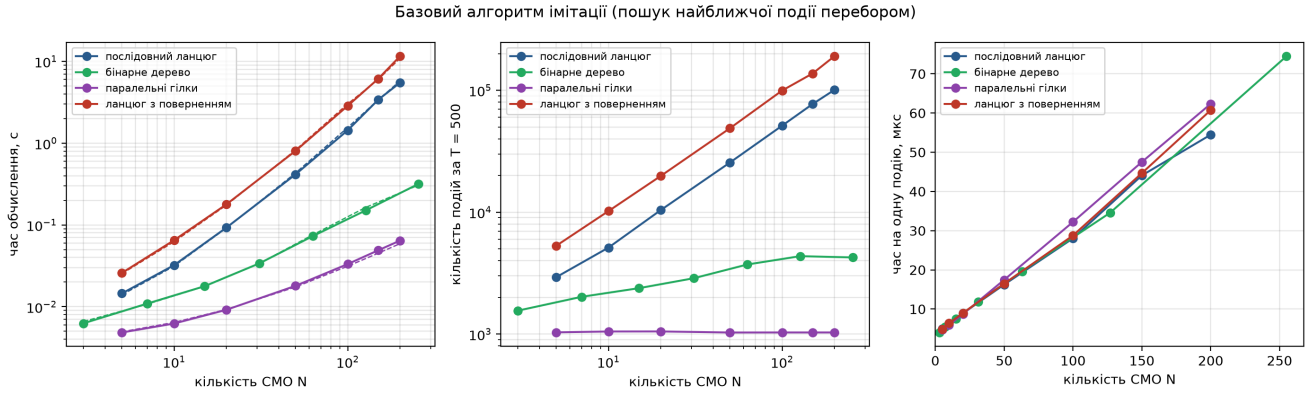


Рисунок 2 – Час обчислення, кількість подій і час на одну подію (базовий алгоритм; штрихові лінії – модель вартості)

Кількість подій зростає лінійно з N і збігається з оцінкою $\lambda \cdot T \cdot (N + 1)$ (табл. 1). Час обчислення при цьому зростає швидше: при збільшенні N від 5 до 200 (у 40 разів) він зріс у 378 разів, а час на одну подію – з 5,0 до 54,5 мкс (рис. 2, синя лінія). Показник степеня в залежності $t \sim N^b$, оцінений за трьома найбільшими N, дорівнює $b = 1{,}95$, тобто час обчислення зростає приблизно квадратично.

Завдання 3:

Алгоритм імітації (метод `Model.simulate`) на кожній події виконує такі дії над списком з N + 2 елементів (N систем, CREATE і DISPOSE):

- пошук моменту найближчої події – мінімум по всіх елементах: N + 2 порівнянь;
- накопичення статистики за інтервал між подіями – виклик `do_statistics` для кожного елемента та підрахунок кількості вимог у системі по всіх N системах: 2N + 2 операцій;
- просування поточного часу кожного елемента: N + 2 присвоювань;
- пошук елементів, подія яких припадає на поточний момент: N + 2 порівнянь;

- виконання самої події (`out_act`, вибір маршруту, `in_act` наступного елемента) – кількість операцій не залежить від N для одного маршруту виходу.

Отже, вартість однієї події лінійна за кількістю елементів: $c_0 + c_1 \cdot (N + 2)$, де c_0 – вартість дій, що не залежать від N, c_1 – вартість обробки одного елемента в циклах. Кількість подій за час T у сталому режимі дорівнює кількості вимог λT, помноженій на кількість подій на одну вимогу. Якщо вимога в середньому проходить L систем, то

$$E = \lambda T\,(1 + L), \quad t = E \cdot (c_0 + c_1(N + 2)) = \lambda T\,(1 + L)(c_0 + c_1(N + 2)).$$

Для послідовного ланцюга L = N, тому кількість подій пропорційна N + 1, а час обчислення $t = \lambda T(N + 1)(c_0 + c_1(N + 2)) = O(\lambda T \cdot N^2)$. Пам’ять алгоритму – O(N): список елементів і стан кожної системи; кількість вимог, що одночасно перебувають у моделі, також пропорційна N.

Константи моделі вартості оцінено методом найменших квадратів за всіма замірами базового алгоритму: c_0 = 2,89 мкс, c_1 = 269 нс. Модель описує всі виміряні значення з відносною похибкою не більше 8,7 % (штрихові лінії на рис. 2 практично збігаються з виміряними), а експериментальний показник степеня 1,95 близький до теоретичного 2. За великих N майже весь час іде на перебір елементів ($c_1(N + 2) \gg c_0$): за N = 200 на нього припадає близько 95 % часу обробки події.

Завдання 4:

Для оцінки впливу розгалужень розглянуто ще три структури мережі з тим самим вхідним потоком і часом обслуговування (рис. 1): бінарне дерево з $N = 2^d - 1$ систем, у якому після кожної системи вимога з ймовірністю 0,5 переходить в одну з двох наступних; паралельні гілки, де вимога з ймовірністю 1/N потрапляє в одну з N систем і виходить; ланцюг з поверненням, у якому після останньої системи вимога з ймовірністю 0,5 повертається в першу. Розгалуження реалізовано маршрутами за ймовірністю універсального алгоритму.

Згідно з моделлю вартості з завдання 3, структура мережі впливає на складність через середню довжину маршруту L(N) – кількість систем, яку проходить одна вимога:

Таблиця 2 – Довжина маршруту і теоретична складність базового алгоритму

Структура	L(N)	Подій E	Складність t(N)
послідовний ланцюг	N	$\lambda T(N + 1)$	$O(N^2)$
ланцюг з поверненням (p = 0,5)	$N / (1 - p) = 2N$	$\lambda T(2N + 1)$	$O(N^2 / (1 - p))$
бінарне дерево	$\log_2(N + 1)$	$\lambda T(1 + \log_2(N + 1))$	O(N log N)
паралельні гілки	1	$2\lambda T$	O(N)

Таблиця 3 – Мережі з розгалуженими маршрутами, базовий алгоритм

Структура	N	Подій E	Оцінка	Час, с	мкс/подію	Модель, с
ланцюг з поверненням	5	5300	5500	0,0259	4,88	0,0253
ланцюг з поверненням	20	19814	20500	0,1783	9,00	0,1746
ланцюг з поверненням	50	48732	50500	0,8031	16,48	0,8228
ланцюг з поверненням	100	99450	100500	2,8614	28,77	3,0172
ланцюг з поверненням	200	190132	200500	11,5480	60,74	10,8844
бінарне дерево	7	2021	2000	0,0109	5,40	0,0107
бінарне дерево	31	2871	3000	0,0339	11,81	0,0338
бінарне дерево	63	3720	3500	0,0731	19,64	0,0758
бінарне дерево	127	4355	4000	0,1506	34,58	0,1638
бінарне дерево	255	4252	4500	0,3167	74,48	0,3063
паралельні гілки	5	1033	1000	0,0048	4,68	0,0049
паралельні гілки	20	1051	1000	0,0092	8,72	0,0093
паралельні гілки	50	1030	1000	0,0180	17,46	0,0174
паралельні гілки	100	1031	1000	0,0332	32,22	0,0313
паралельні гілки	200	1031	1000	0,0643	62,35	0,0590

Експерименти (табл. 3, рис. 2) підтверджують теоретичні оцінки. Кількість подій для всіх структур збігається з $\lambda T(1 + L(N))$: для ланцюга з поверненням вона вдвічі більша, ніж для ланцюга, для дерева зростає логарифмічно, а для паралельних гілок не залежить від N. Час на одну подію в усіх структурах однаковий і лінійно зростає з N (рис. 2, праворуч), бо базовий алгоритм перебирає всі елементи незалежно від маршрутів. Експериментальні показники степеня: послідовний ланцюг – 1,95, бінарне дерево – 1,05, паралельні гілки – 0,95, ланцюг з поверненням – 2,00 (теорія: 2, 1 з логарифмічним множником, 1, 2). Модель вартості з тими самими константами c_0, c_1 описує всі чотири структури.

Розгалуження впливає на складність і безпосередньо, через вибір маршруту: щоб вибрати один з k маршрутів за ймовірністю, алгоритм перебирає накопичені ймовірності, тобто витрачає O(k) операцій на кожен вихід із точки розгалуження. У бінарному дереві k = 2, і цей доданок непомітний. У паралельній структурі

CREATE має k = N маршрутів, і для базового алгоритму ця вартість додається до перебору елементів, не змінюючи порядку O(N) на подію.

Оскільки за великих N основну частину часу займає перебір елементів, а не самі події, алгоритм було модифіковано (клас `FastModel`). Моменти наступних подій зберігаються в календарі подій – двійковій купі: найближча подія береться з вершини купи, а елемент, у якого змінився момент наступної події, додається в купу за O(log N) операцій (застарілі записи пропускаються під час вибору). Статистику накопичує лише той елемент, стан якого змінюється, за час від попередньої зміни його стану, а поточний час передається тільки елементам, що беруть участь у події. За однакового зерна генератора обидва алгоритми дають ідентичні траєкторії моделі (ті самі події та результати), змінюється лише час обчислення. Вартість однієї події становить O(log N), тому

$$t = \lambda T\,(1 + L(N)) \cdot (c_0 + c_1 \log_2(N + 2))$$

і складність стає O(N log N) для ланцюга та ланцюга з поверненням, O(log² N) для дерева. Для паралельних гілок вартість події визначає вже вибір маршруту з N варіантів, тож складність лишається O(N).

Таблиця 4 – Час обчислення алгоритму з календарем подій

Структура	N	Подій E	Час, с	мкс/подію
послідовний ланцюг	5	2927	0,0093	3,18
послідовний ланцюг	50	25501	0,0841	3,30
послідовний ланцюг	200	100649	0,3276	3,25
послідовний ланцюг	1000	497864	1,8064	3,63
ланцюг з поверненням	5	5300	0,0249	4,70
ланцюг з поверненням	50	48732	0,1686	3,46
ланцюг з поверненням	200	190132	0,6830	3,59
ланцюг з поверненням	1000	919785	3,5275	3,84
бінарне дерево	7	2021	0,0062	3,09
бінарне дерево	63	3720	0,0117	3,16
бінарне дерево	255	4252	0,0142	3,33
бінарне дерево	1023	5810	0,0176	3,02
паралельні гілки	5	1033	0,0029	2,83
паралельні гілки	50	1030	0,0042	4,03
паралельні гілки	200	1031	0,0083	8,01
паралельні гілки	1000	1033	0,0289	27,95

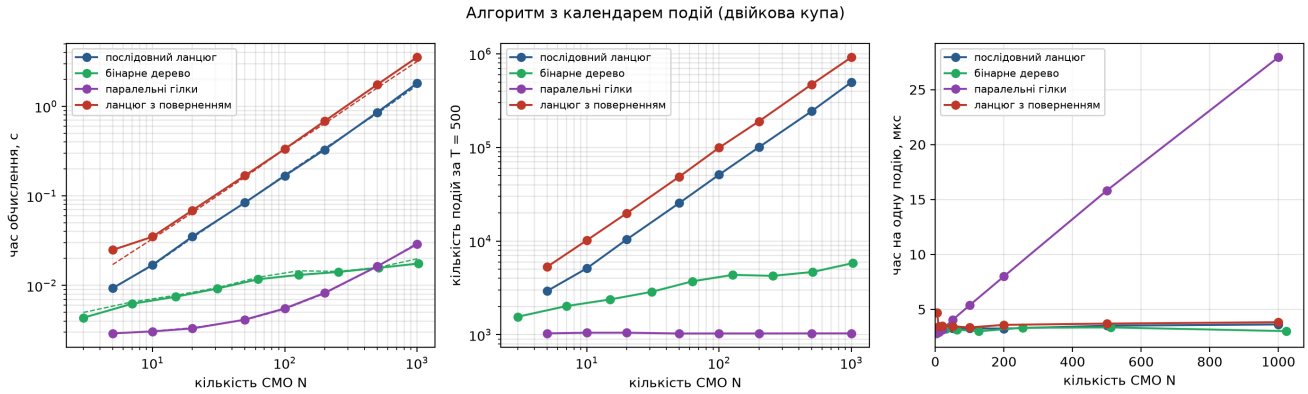


Рисунок 3 – Алгоритм з календарем подій: час обчислення, кількість подій і час на одну подію

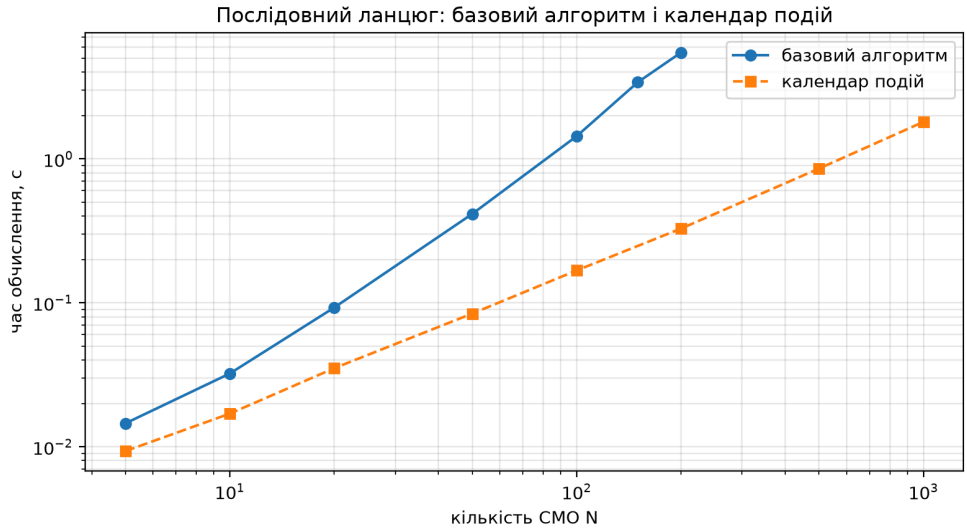


Рисунок 4 – Послідовний ланцюг: порівняння алгоритмів

Для алгоритму з календарем подій час на одну подію майже сталий (близько 3–4 мкс) для ланцюга, дерева і ланцюга з поверненням навіть за N = 1000 (табл. 4, рис. 3), а показники степеня послідовний ланцюг – 1,06, бінарне дерево – 0,15, паралельні гілки – 0,78, ланцюг з поверненням – 1,02 відповідають теоретичним оцінкам. Для ланцюга з N = 200 час обчислення зменшився з 5,48 до 0,33 с, тобто в 17 разів (рис. 4). Модель вартості з c_0 = 3,15 мкс і c_1 = 29 нс описує заміри для ланцюга, дерева і ланцюга з поверненням; найбільше відхилення мають найкоротші прогони (кілька мілісекунд), де помітна похибка вимірювання часу. Логарифмічний доданок малий порівняно з c_0, тому на практиці вартість події майже не залежить від N. Для паралельних гілок час на подію зростає лінійно (модель $t = E \cdot (c_0 + c_1 \cdot N/2)$ з похибкою до 2,6 %) – тепер це видно чітко, бо перебір елементів більше не маскує вартість вибору маршруту.

Висновок

У роботі побудовано модель масового обслуговування з N послідовно з'єднаних систем, у якій N є параметром, а кількість подій становить N + 1, та виконано експериментальну й теоретичну оцінку складності алгоритму імітації.

Час обчислення алгоритму визначається добутком кількості подій на вартість обробки однієї події. Базовий алгоритм на кожній події перебирає всі N + 2 елементи (пошук найближчої події, накопичення статистики, просування часу), тому вартість події лінійна за N. Для послідовного ланцюга кількість подій також пропорційна N, і складність становить $O(N^2)$: експериментальний показник степеня 1,95, а модель вартості $t = \lambda T(N + 1)(c_0 + c_1(N + 2))$ описує заміри з похибкою до 9 %.

Розгалуження маршрутів змінює складність через середню довжину маршруту вимоги L(N), яка задає кількість подій: повернення збільшують її в $1/(1 - p)$ разів і зберігають квадратичну складність, розгалуження на дерево скорочує маршрут до $\log_2 N$ і дає $O(N \log N)$, паралельні гілки – до однієї системи і дають O(N). Вартість однієї події від структури не залежить, лише у точках розгалуження додається вибір маршруту O(k), помітний при великій кількості гілок k. Отже, чим коротший маршрут вимоги порівняно з кількістю систем, тим менша складність імітації.

Заміна перебору елементів календарем подій у вигляді двійкової купи і накопичення статистики лише в елементах, що змінюють стан, знижує вартість події до O(log N): для ланцюга показник степеня зменшився з 1,95 до 1,06, а час обчислення за N = 200 зменшився в 17 разів при ідентичних результатах моделювання.